

Алгоритм проталкивания предпотока (Push-Relabel)

Антонов Кирилл
Б01-411

Курс «Алгоритмы и структуры данных»

Проблема

Задача о максимальном потоке заключается в том, чтобы найти наибольший объём жидкости (или данных), который можно передать из истока в сток с учётом пропускных способностей рёбер. Классические алгоритмы (Форд-Фалкерсон, Диниц) на каждом шаге ищут увеличивающий путь от истока до стока. В больших графах (миллионы узлов) каждый такой поиск требует обхода значительной части графа, что приводит к высокому времени работы.

Идея

Можно отказаться от поиска полных путей и работать локально: проталкивать поток по одному ребру за раз, временно нарушая закон сохранения потока в узлах.

Вывод

Для больших плотных графов требуется алгоритм без глобального поиска путей — таким является алгоритм проталкивания предпотока (push-relabel).

Постановка задачи о максимальном потоке

Что дано

Задан ориентированный граф $G = (V, E)$ с выделенными вершинами: истоком s и стоком t . Каждому ребру (u, v) приписана пропускная способность $c(u, v) \geq 0$ — максимальный объём потока, который может пройти по этому ребру.

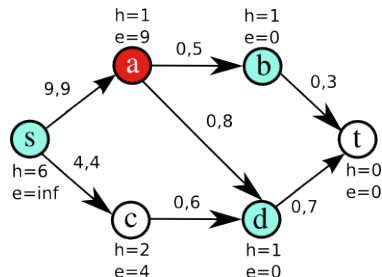
Что нужно найти

Функцию $f : V \times V \rightarrow \mathbb{R}_{\geq 0}$, называемую **потоком**, такую что:

- ▶ $0 \leq f(u, v) \leq c(u, v)$ для всех (u, v) — ограничение пропускной способности
- ▶ $\sum_v f(u, v) = \sum_v f(v, u)$ для всех $u \notin \{s, t\}$ — закон сохранения потока (сколько втекло, столько и вытекло)

Цель: максимизировать суммарный поток, выходящий из истока:

$$\text{maximize} \quad \sum_v f(s, v)$$



Пример графа с пропускными способностями

Ключевая идея: предпоток (preflow)

Поток

В классическом определении поток в промежуточных узлах должен сохраняться: сколько втекло, столько и вытекло. Это глобальное условие, которое алгоритмы вроде Диница поддерживают на каждом шаге.

Предпоток

Алгоритм проталкивания ослабляет это требование: в узлах может временно накапливаться избыток $e(u) \geq 0$, где

$$e(u) = \sum_v f(v, u) - \sum_v f(u, v)$$

Узлы с положительным избытком называются **активными**. Именно они обрабатываются алгоритмом.

Зачем это нужно

Ослабление условий позволяет работать локально: мы не ищем пути от истока до стока, а просто проталкиваем поток из активных узлов к соседям. Глобальный баланс восстановится сам собой в конце алгоритма, когда активных узлов не останется.

Важно

Алгоритм начинает работу с предпотока (есть избытки) и заканчивает настоящим потоком (избытков нет).

Основные понятия: высота (height)

Высота вершины $h(u)$

- ▶ Искусственная «гравитация». В аналогии с водой и трубами: вода течёт только **вниз**
- ▶ Целое неотрицательное число. Изначально: $h(s) = |V|$ (больше всех), $h(t) = 0$ (ниже всех), для остальных $h = 0$
- ▶ Высота может только увеличиваться (операция relabel)

Правило течения

Вода может быть протолкнута из u в v (операция push), только если:

- ▶ Остаточная пропускная способность $c_f(u, v) > 0$
- ▶ $h(u) = h(v) + 1$ (течёт ровно на 1 уровень вниз)

Как используются высоты

- ▶ Направляют поток к стоку: сток имеет минимальную высоту 0, поэтому поток стремится вниз
- ▶ Предотвращают заикливание: высоты только растут, поэтому вода не может бесконечно циркулировать
- ▶ Активный узел ($e(u) > 0$) либо проталкивает поток (если есть сосед на $h - 1$), либо повышает свою высоту (relabel)

Инициализация

- ▶ $h(s) = |V|$ (самый высокий), $h(t) = 0$ (самый низкий), остальные также $h = 0$
- ▶ Насыщаем все рёбра из истока: проталкиваем максимально возможный поток $c(s, v)$
- ▶ У истока образуется отрицательный избыток (дефицит), у его соседей — положительный избыток

Операция Push (проталкивание)

Условия: узел u имеет избыток ($e(u) > 0$), ребро $u \rightarrow v$ имеет остаточную пропускную способность ($c_f > 0$), и $h(u) = h(v) + 1$ (течёт ровно на 1 вниз)

Действие: пересылаем $\min(e(u), c_f(u, v))$ единиц потока из u в v . После этого u может перестать быть активным, а v — стать активным.

Операция Relabel (подъём)

Условия: узел u имеет избыток ($e(u) > 0$), но нет соседа, подходящего для push (все соседи с $c_f > 0$ имеют высоту $\geq h(u)$)

Действие: увеличиваем высоту u до значения, чтобы появился сосед на 1 ниже:
 $h(u) = 1 + \min\{h(v) \mid c_f(u, v) > 0\}$. После подъёма становится возможен push.

Инварианты

- ▶ **Направление потока:** push только при $h(u) = h(v) + 1$ (строго вниз)
- ▶ **Монотонность высот:** $h(u)$ только увеличивается (relabel)
- ▶ **Исток и сток:** $h(s) = |V|$, $h(t) = 0$ всегда

Ограничение на число операций

- ▶ Каждый relabel увеличивает $h(u)$, максимум $h(u) \leq 2|V| \rightarrow O(|V|^2)$ релейблов
- ▶ Насыщающий push уменьшает суммарный избыток
- ▶ Ненасыщающий push переносит избыток ближе к стоку

Итог

Алгоритм всегда завершается, так как число операций ограничено. После завершения избытков нет — получен максимальный поток.

Где используется максимальный поток

- ▶ **Интернет-маршрутизация:** распределение трафика по каналам
- ▶ **Логистика:** максимальный объём груза по сети дорог
- ▶ **Сегментация изображений:** разрезание графа (Graph Cut)
- ▶ **Распределение задач:** назначение работников на операции

Почему выбирают push-relabel

- ▶ Локальность работы → хорошая кэш-эффективность
- ▶ Легко параллелится (можно толкать из нескольких узлов одновременно) + Один из самых быстрых на плотных графах

Базовая версия (без эвристик)

$$O(V^2 \cdot E)$$

Достигается за счёт того, что каждый relabel увеличивает высоту узла (не более $O(V)$ раз на узел), а каждый push может насытить ребро не более $O(V)$ раз.

Улучшенные версии

- ▶ **FIFO (очередь):** $O(V^3)$ — обрабатывает активные узлы в порядке очереди
- ▶ **Highest-Label + Gap эвристики (HLPP):** $O(V^2\sqrt{E})$ — выбирает узел с максимальной высотой, пропускает недостижимые высоты

Число операций

- ▶ Relabel: $O(V^2)$ (каждый узел может быть поднят не более $2V$ раз)
- ▶ Push: $O(V^2 \cdot E)$ в базовой версии, меньше в HLPP

Подробные доказательства оценок приведены в литературе: НИУ ИТМО: Алгоритм проталкивания предпотока

Теоретическая сложность

Алгоритм	Сложность	Особенности
Форд-Фалкерсон	$O(E \cdot f_{\max})$	Простой, но может работать очень долго при больших потоках
Диниц	$O(V^2 E)$	Хорош на разреженных графах, использует BFS + DFS
Push-Relabel (базовый)	$O(V^2 E)$	Работает локально, но без эвристик медленнее
Push-Relabel (HLPP)	$O(V^2 \sqrt{E})$	С эвристиками Highest-Label и Gap — самый быстрый на плотных графах

Ключевые различия

- ▶ **Диниц** требует глобального поиска увеличивающих путей (BFS/DFS), что даёт преимущество на разреженных графах
- ▶ **Push-Relabel** работает локально (только push/relabel), не ищет пути целиком — выигрывает на плотных графах, где путей слишком много
- ▶ **HLPP-эвристики** (Highest-Label + Gap) значительно ускоряют работу на практике, приближая сложность к $O(V^2 \sqrt{E})$

Что выбрали для бенчмарков

Для сравнения взяты эталонные реализации Dinic и Push-Relabel (HLPP) из библиотеки KACTL — обе оптимизированы и широко используются в соревновательном программировании.

Тестовая среда

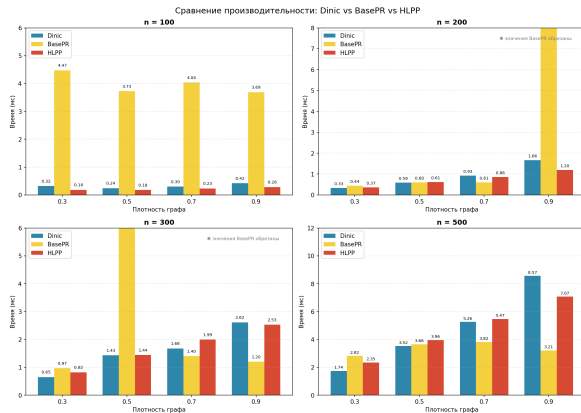
- ▶ Intel i5-12450h, 16GB RAM, Manjaro Linux, g++ -O2
- ▶ 5 различных графов на конфигурацию, медиана

Результаты

- ▶ **HLPP** — стабильно быстр на плотных графах (200/0.9: 1.16 мс vs 1.60 у Dinic)
- ▶ **BasePR** — быстр на 500/0.9 (3.16 мс), но есть выбросы (300/0.5: 136.49 мс)
- ▶ **Dinic** — хорош на разреженных графах

Вывод

HLPP подтверждает оценку $O(V^2\sqrt{E})$ и опережает Dinic на плотных графах.



* — значения BasePR обрезаны (см. таблицу)

Основные массивы

```
int n;  
vector<vector<int>> cap;    // residual capacity  
vector<int> excess;        // excess flow  
vector<int> height;        // height  
vector<vector<int>> adj;    // adjacency list
```

Инициализация (насыщение рёбер из истока)

```
height[s] = n;  
for (int v : adj[s]) {  
    cap[s][v] = initial_cap[s][v];  
    excess[v] = cap[s][v];  
    excess[s] -= cap[s][v];  
    cap[v][s] = 0;  
}
```

Операция push — проталкивание потока

```
void push(int u, int v) {  
    int flow = min(excess[u], cap[u][v]);  
    cap[u][v] -= flow;  
    cap[v][u] += flow;  
    excess[u] -= flow;  
    excess[v] += flow;  
}
```

Избыток узла u перенаправляется в узел v . Обратное ребро получает остаточную пропускную способность.

Операция relabel — подъём высоты

```
void relabel(int u) {  
    int min_h = INF;  
    for (int v : adj[u])  
        if (cap[u][v] > 0)  
            min_h = min(min_h, height[v]);  
    height[u] = min_h + 1;  
}
```

Высота узла становится на единицу больше минимальной высоты соседа, в который можно протолкнуть поток.

Основная литература

- ▶ **CP-Algorithms**: <https://cp-algorithms.com/graph/push-relabel.html>
- ▶ **НИУ ИТМО**: Алгоритм проталкивания предпотока
- ▶ **UT Austin**: https://iss.oden.utexas.edu/?p=projects/galois/analytics/preflow_push

Исходный код

- ▶ GitHub репозиторий: <https://github.com/navernokeril/push-relabel>
- ▶ Для бенчмарков использована реализация Push-Relabel (HLPP) из библиотеки KACTL
- ▶ Исходники LaTeX, примеры кода, бенчмарки и графики доступны в репозитории

Спасибо за внимание!